

Task 5 : Working with SLURM - Scripting & Kubeflow

Date: 27-03-2026

PART 1 — SLURM

Procedure

The purpose of this study was to analyze how job scheduling is performed using the SLURM workload manager in a container environment. To perform the experiment, the SLURM cluster was created based on the Docker platform that comprised a controller node as well as compute nodes. In order to test SLURM functionality, a batch job consisting of matrix addition implemented with Python was run.

Step-by-Step Explanation

The SLURM cluster setup was done using Docker-based cluster setup.

```
git clone https://github.com/giovtorres/slurm-docker-cluster.git
cd slurm-docker-cluster
docker compose up -d
```

These clusters running were checked by doing the following commands:

```
docker ps
docker exec -it slurmctld bash
sinfo
```

A simple Python script to add two matrices:

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

result = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]
print(result)
```


PERSONAL

Ctrl+K

FPS N/A | GPU 0% 51°C | CPU 5% | LAT N/A

Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers [Give feedback](#)

Container CPU usage i

0.36% / 1600% (16 CPUs available)

Container memory usage

289.13MB / 7.39GB

Only show running containers

☐		Name	Container ID	Image	Port(s)
☐	☐	charming_easley	9383cb167579	sid110/hume-rainfall-app	
☐	☐	amazing_nightingale	452b3aeb7293	nvidia/pytorch:26.01-py3	

Terminal

```

Complete!
[root@slurmctld data]# nano matrix.py
[root@slurmctld data]# matrix.slurm
bash: matrix.slurm: command not found
[root@slurmctld data]# nano matrix.py
[root@slurmctld data]# nano matrix.py
[root@slurmctld data]# nano matrix.slurm
[root@slurmctld data]# sbatch matrix.slurm
Submitted batch job 1
[root@slurmctld data]# squeue
          JOBID PARTITION    NAME    USER  ST       TIME  NODES NODELIST(REASON)
[root@slurmctld data]# watch squeue
[root@slurmctld data]# cat result.txt
Job Started
Matrix Addition using SLURM
Result:
[6, 8]
[10, 12]
Job Completed
[root@slurmctld data]#

```

Upgrade plan

Engine running

RAM 6.67 GB

CPU 0.12%

Disk: 123.30 GB used (limit 1006.85 GB)

A

PERSONAL

afeefahmed

What's new

Account Settings

Upgrade

Sign out

Show charts

⋮

⋮

Windo...

⋮

⋮

Terminal

v4.71.0

Fig 36: SLURM Job Output

60

A batch script with SLURM commands was developed as follows:

```
#!/bin/bash
#SBATCH --job-name=matrix_add
#SBATCH --output=result.txt
python3 matrix.py
```

The script was successfully run:

```
sbatch matrix.slurm
cat result.txt
```

As seen from the result, the experiment succeeded as SLURM was able to schedule and execute the command in question.

PART 2 — Kubeflow

Procedure

The goal of the experiment was workflow automation with Kubeflow Pipelines. Thus, a matrix addition component had to be created and compiled into a YAML file describing the workflow process.

Step-by-Step Explanation

Firstly, we installed Kubeflow Pipelines SDK to allow us to define our workflows:

```
pip install kfp
```

PERSONAL

Ctrl+K

A

Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers [Give feedback](#)

Container CPU usage

1.24% / 1600% (16 CPUs available)

Container memory usage

169.53MB / 7.39GB

☐ Only show running containers

	Name	Container ID	Image	Port(s)
	Terminal			

```

PS C:\Users\afeef> notepad pipeline.py
PS C:\Users\afeef> python pipeline.py
C:\Users\afeef\pipeline.py:4: FutureWarning: The default base_image used by the @dsl.component decorator will switch from 'python:3.11' to 'python:3.12' on Oct 1, 2027. To ensure your existing components work with versions of the KFP SDK released after that date, you should provide an explicit base_image argument and ensure your component works as intended on Python 3.12.
  @dsl.component
PS C:\Users\afeef> dir

Directory: C:\Users\afeef

Mode                LastWriteTime         Length Name
----                -
d-----          4/11/2026   7:18 PM             .android
d-----          4/11/2026  12:57 AM             .antigravity
d-----         11/6/2025  11:46 PM             .arduinoIDE
d-----          8/16/2025   1:26 PM             .cache
d-----          1/27/2026  10:16 AM             .cagent
d-----           8/2/2025   7:31 PM             .chocolatey
d-----          11/7/2025  12:34 AM             .codeium
d-----          1/27/2026  10:16 AM             .config
d-----          2/18/2026   9:32 AM             .copilot
d-----          5/2/2026   3:32 AM             .docker
          
```

Upgrade plan

Engine running

RAM 1.60 GB CPU 0.38% Disk: 123.30 GB used (limit 1006.85 GB)

Terminal

v4.71.0

A

PERSONAL

afeefahmed

What's new

Account Settings

Upgrade

Sign out

Show charts

Windo...

Windo...

Fig 37:Kubeflow Pipeline Script

62

A script in Python was developed to implement a reusable module to perform matrix addition. This involved building logic for matrix addition:

```
from kfp import dsl
from kfp.compiler import Compiler

@dsl.component
def matrix_addition():
    print("Matrix Addition using Kubeflow")

    A = [[1, 2], [3, 4]]
    B = [[5, 6], [7, 8]]

    result = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]

    print("Result:")
    for row in result:
        print(row)

@dsl.pipeline(name="matrix-addition-pipeline")
def pipeline():
    matrix_addition()
```

Execution:

```
python pipeline.py
```

The resulting pipeline.yaml indicated that the pipeline was successfully created..

Task 6 : Using Jupyter Notebook and PyTorch for Tensor Operations and Data-Centric Application

Date: 08-04-2026

Experiment Procedure

This experiment was designed to understand tensor operations and create a data-centric application using Jupyter Notebook and PyTorch. The following steps were executed to complete the task.

Step-by-Step Explanation

Installation of required libraries and opening of notebook.
Performance of tensor operations using Python script.
Creation of data-centric application using linear regression model.
Experiment Steps Explanation

In order to perform the experiment, all the required libraries were installed and Jupyter Notebook was opened:

```
pip install notebook torch torchvision  
jupyter notebook
```

After opening the notebook, two tensors were created and several operations like addition, subtraction, element-wise multiplication, and matrix multiplication were executed on them.

```
import torch  
  
a = torch.tensor([[1, 2], [3, 4]])  
b = torch.tensor([[5, 6], [7, 8]])  
  
print("Addition:\n", a + b)  
print("Subtraction:\n", a - b)  
print("Element-wise Multiplication:\n", a * b)  
print("Matrix Multiplication:\n", torch.matmul(a, b))
```

Further, the data-centric application was prepared by creating a dataset containing linearly related variables and defining a linear model using PyTorch.

```
import torch.nn as nn  
import torch.optim as optim  
  
x = torch.tensor([[1.0], [2.0], [3.0], [4.0]])  
y = torch.tensor([[3.0], [5.0], [7.0], [9.0]])  
  
model = nn.Linear(1, 1)
```

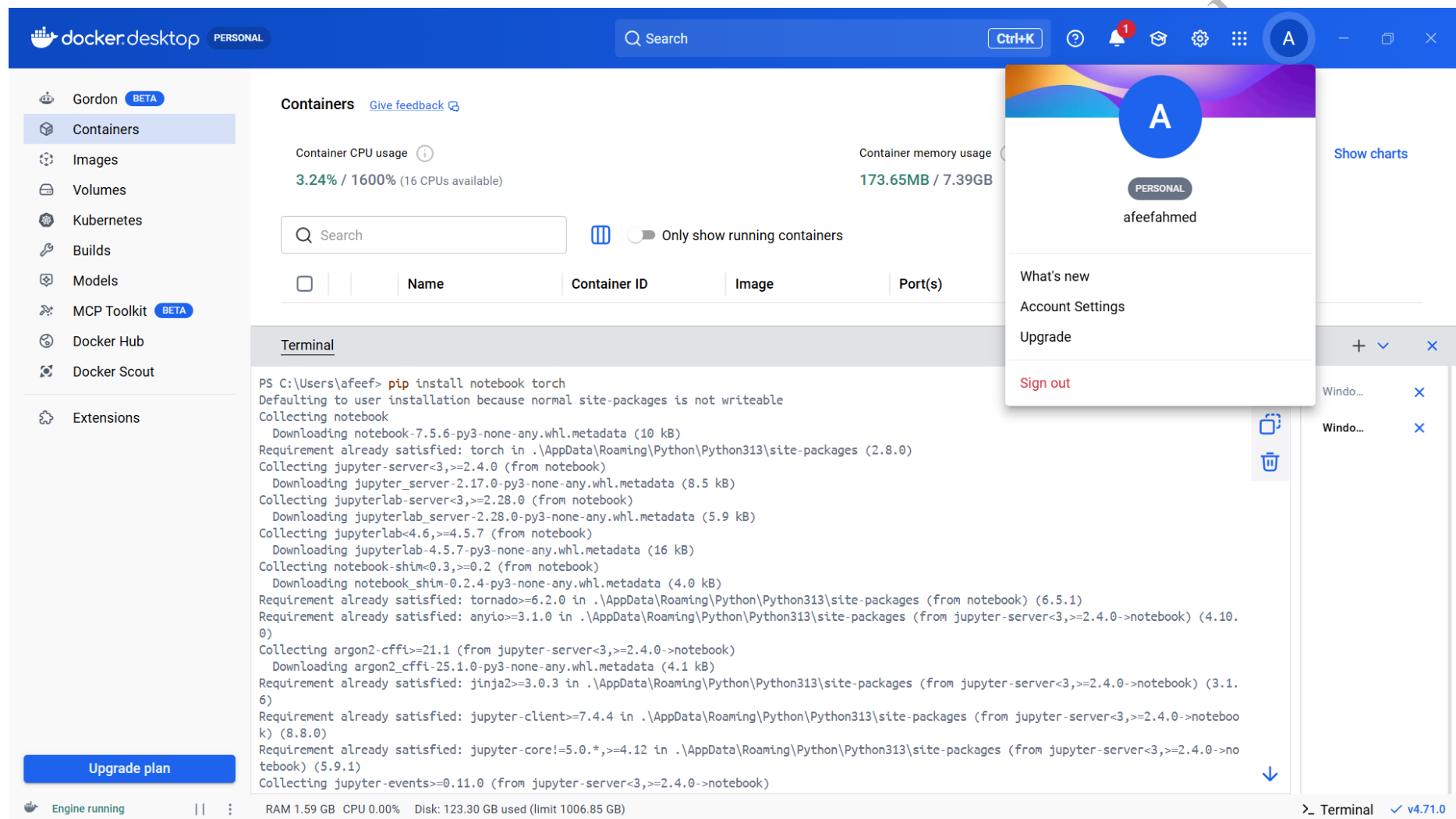
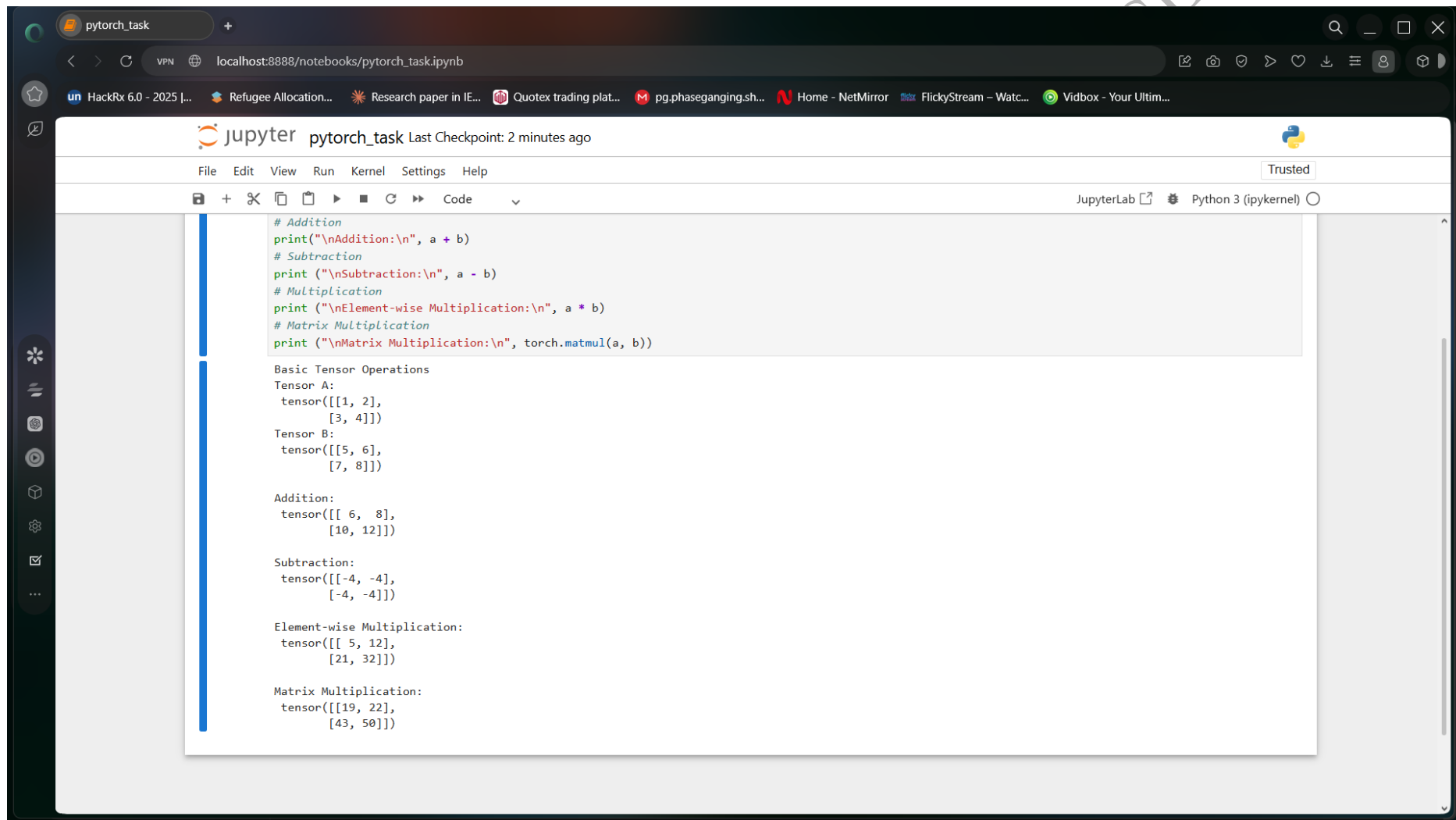


Fig 39:Jupyter Installation Process



The screenshot shows a JupyterLab window titled "pytorch_task" with a "Last Checkpoint: 2 minutes ago" status. The interface includes a top toolbar with icons for file operations, a menu bar with "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help", and a "Trusted" security indicator. The main area contains a code editor with the following Python code:

```
# Addition
print("\nAddition:\n", a + b)
# Subtraction
print ("\nSubtraction:\n", a - b)
# Multiplication
print ("\nElement-wise Multiplication:\n", a * b)
# Matrix Multiplication
print ("\nMatrix Multiplication:\n", torch.matmul(a, b))
```

Below the code editor, the output of the operations is displayed:

Basic Tensor Operations
Tensor A:
tensor([[1, 2],
 [3, 4]])
Tensor B:
tensor([[5, 6],
 [7, 8]])

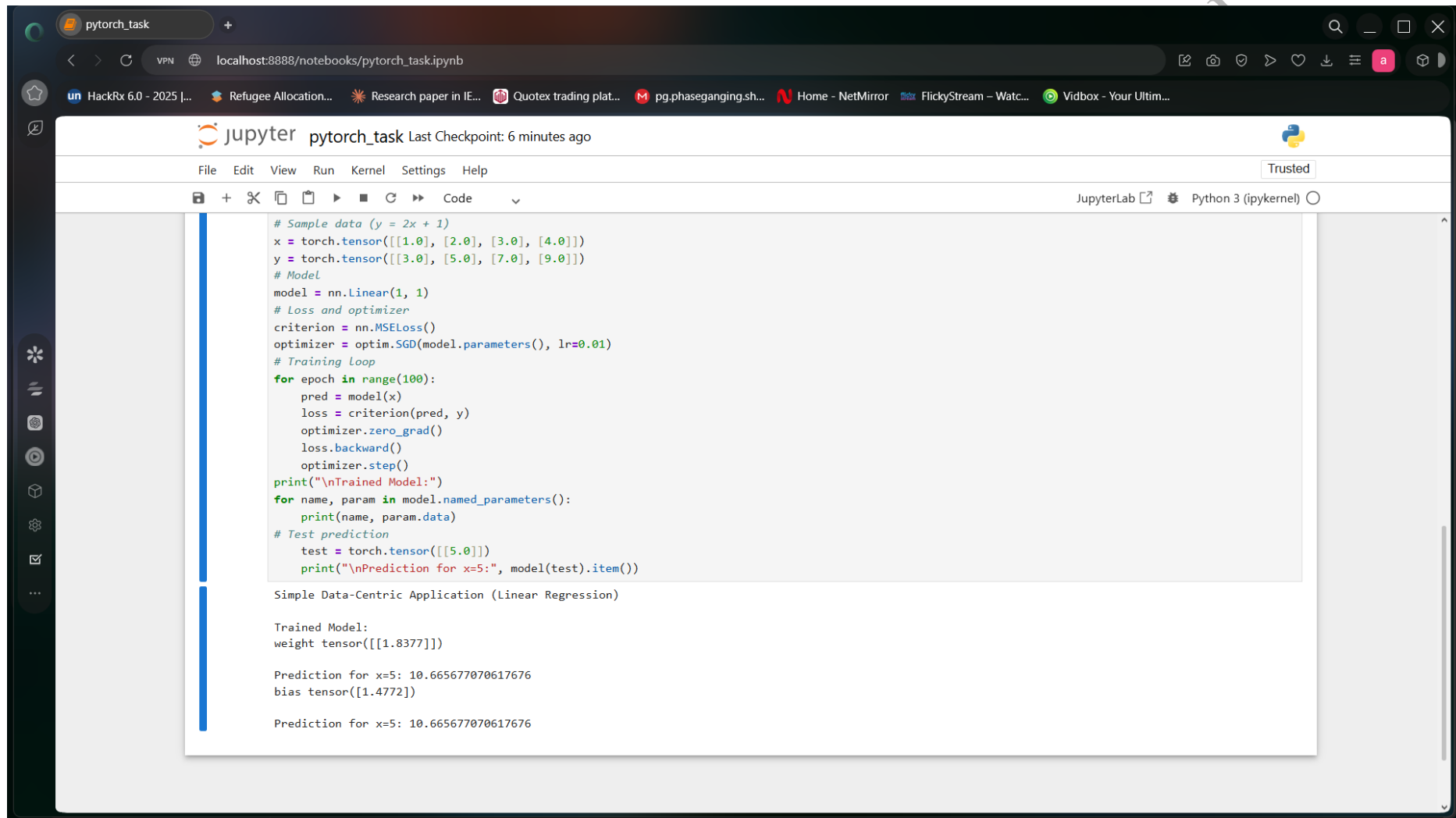
Addition:
tensor([[6, 8],
 [10, 12]])

Subtraction:
tensor([[-4, -4],
 [-4, -4]])

Element-wise Multiplication:
tensor([[5, 12],
 [21, 32]])

Matrix Multiplication:
tensor([[19, 22],
 [43, 50]])

Fig 40:Jupyter Tensor Output



```
# Sample data (y = 2x + 1)
x = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([[3.0], [5.0], [7.0], [9.0]])
# Model
model = nn.Linear(1, 1)
# Loss and optimizer
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
# Training Loop
for epoch in range(100):
    pred = model(x)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
print("\nTrained Model:")
for name, param in model.named_parameters():
    print(name, param.data)
# Test prediction
test = torch.tensor([[5.0]])
print("\nPrediction for x=5:", model(test).item())
```

Simple Data-Centric Application (Linear Regression)

Trained Model:
weight tensor([[1.8377]])

Prediction for x=5: 10.665677070617676
bias tensor([1.4772])

Prediction for x=5: 10.665677070617676

Fig 41:Linear Regression Output

After preparing the dataset and defining the model, the next step involved specifying a loss function and an optimizer in order to train the linear regression model iteratively.

```
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

```
for epoch in range(100):
    pred = model(x)
    loss = criterion(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

After training the model using iterative epochs, the final step was to test the trained model using test inputs.

```
test = torch.tensor([[5.0]])
print("Prediction for x=5:", model(test).item())
```

Finally, we checked whether the newly trained linear model gave correct predictions for new inputs.

Task 7 : Testing the GPGPU Computing

Date: 21-04-2026

Procedure

In this experiment, analysis and comparison of CPU-based computation vs GPU-based computation was carried out using PyTorch. The objective was to show the improvements offered by the use of GPUs for parallel computing on large scale tasks.

Step-by-Step Explanation

In order to confirm the availability of a GPU, the following code snippet was executed.

```
import torch

print("GPU Available:", torch.cuda.is_available())
```

Firstly, a large dataset was created. Sorting operation was performed using CPU, and runtime was measured.

```
import time

data = torch.rand(10_000_000)

start = time.time()
torch.sort(data)
cpu_time = time.time() - start

print("CPU Time:", cpu_time)
```

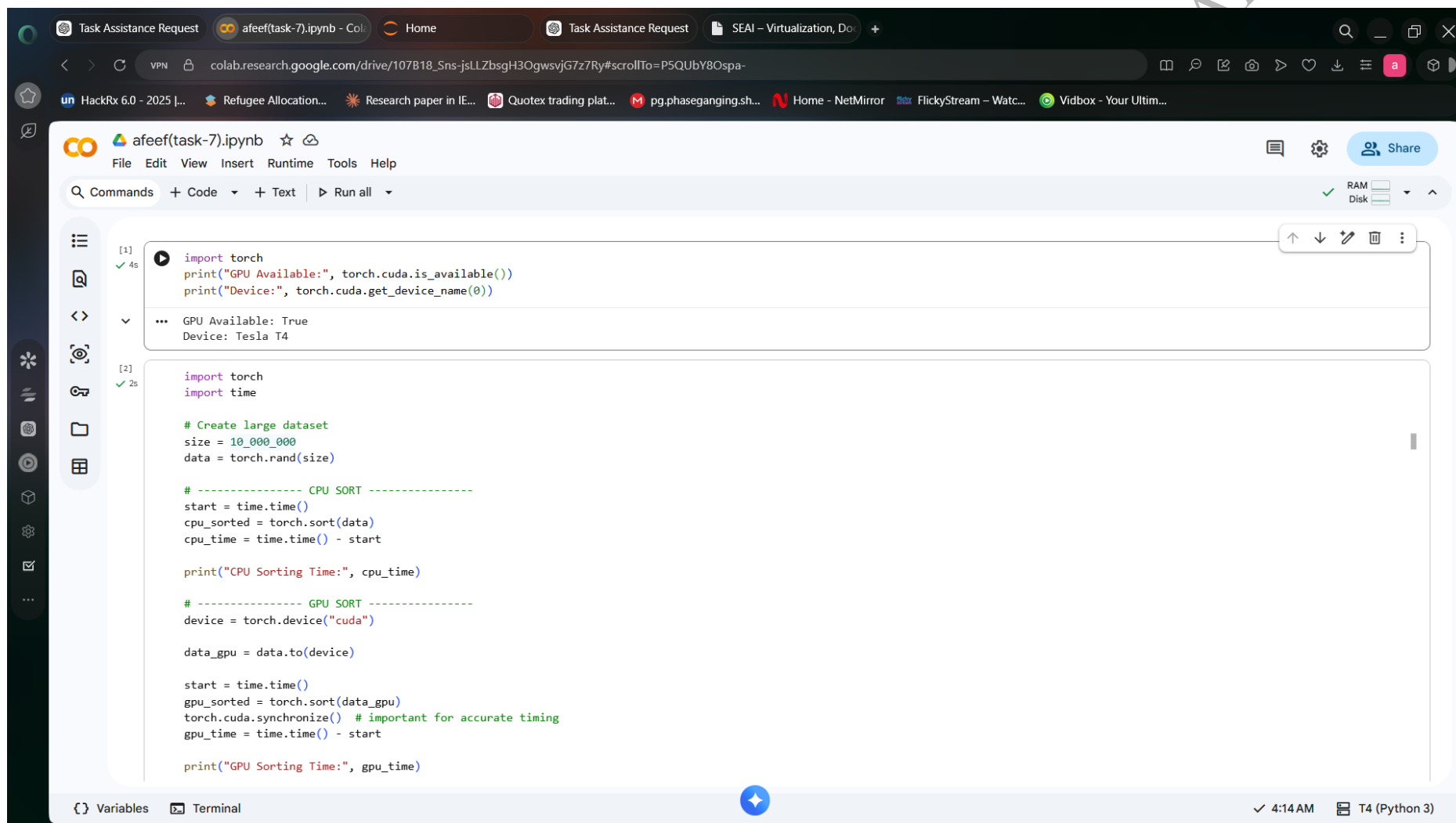


Fig 42:GPU Verification Output

The screenshot displays a Jupyter Notebook environment within a web browser. The browser's address bar shows a Google Drive link. The notebook interface includes a top menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with icons for running code, saving, and other functions. The main area of the notebook contains two code cells. The first cell, labeled [1], contains a single line of Python code: `print("Device:", torch.cuda.get_device_name(0))`. The output for this cell shows "GPU Available: True" and "Device: Tesla T4". The second cell, labeled [2], contains a more complex Python script that creates a large dataset, sorts it using both CPU and GPU, and prints the sorting times. The output for the second cell shows "CPU Sorting Time: 1.619107723236084" and "GPU Sorting Time: 0.06518912315368652". The bottom of the notebook interface shows a status bar with "Variables", "Terminal", and a clock indicating 4:14 AM on a Tesla T4 (Python 3) environment.

```
[1] ✓ 4s
print("Device:", torch.cuda.get_device_name(0))

... GPU Available: True
Device: Tesla T4

[2] ✓ 2s
import torch
import time

# Create large dataset
size = 10_000_000
data = torch.rand(size)

# ----- CPU SORT -----
start = time.time()
cpu_sorted = torch.sort(data)
cpu_time = time.time() - start

print("CPU Sorting Time:", cpu_time)

# ----- GPU SORT -----
device = torch.device("cuda")

data_gpu = data.to(device)

start = time.time()
gpu_sorted = torch.sort(data_gpu)
torch.cuda.synchronize() # important for accurate timing
gpu_time = time.time() - start

print("GPU Sorting Time:", gpu_time)

... CPU Sorting Time: 1.619107723236084
GPU Sorting Time: 0.06518912315368652
```

Fig 43:GPU Sorting Output

Then, the same data was moved to the GPU memory, and the sorting operation was again performed to measure runtime.

```
device = torch.device("cuda")
```

```
data_gpu = data.to(device)
```

```
start = time.time()
```

```
torch.sort(data_gpu)
```

```
torch.cuda.synchronize()
```

```
gpu_time = time.time() - start
```

```
print("GPU Time:", gpu_time)
```

As seen from the results, GPU computations were considerably more efficient compared to CPU for large data sets, which demonstrated the benefits of parallel computations.

Task 8 : Testing CUDA-X Tensor Operations, Mixed Precision, and Quantization-Aware Training using PyTorch

Date: 25-04-2026

Procedure

In the current experiment, we explored GPU programming in PyTorch through advanced CUDA tensor operations, mixed precision, and automatic differentiation.

Step-by-Step Explanation

In the following example, the device was selected as either GPU or CPU based on availability.

```
import torch
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print("Using:", device)
```

Large tensors were created on GPU and matrix multiplication was applied to them.

```
A = torch.randn(4096, 4096, device=device)  
B = torch.randn(4096, 4096, device=device)
```

```
C = A @ B  
print("Matrix multiplication completed")
```

Autodiff capabilities in PyTorch were tested next.

```
x = torch.tensor(2.0, requires_grad=True)  
y = x**3 + 2*x**2 + x
```

```
dy_dx = torch.autograd.grad(y, x, create_graph=True)[0]  
d2y_dx2 = torch.autograd.grad(dy_dx, x)[0]
```

```
print("First derivative:", dy_dx.item())  
print("Second derivative:", d2y_dx2.item())
```

Task Assistance Request afeef(task-8).ipynb - Col Home Task Assistance Request SEAL - Virtualization, Do +

colab.research.google.com/drive/107B18_Sns-jsLLZbSgH3OgwsvjG7z7Ry#scrollTo=VmhmeOptDxi

HackRx 6.0 - 2025 [...] Refugee Allocation... Research paper in IE... Quotex trading plat... pg.phaseganging.sh... Home - NetMirror FlickyStream - Watc... Vidbox - Your Ultim...

afeef(task-8).ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[3] ✓ 0s

```
import torch, time
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using:", device)
if device.type == "cuda":
    print(torch.cuda.get_device_name(0))
```

... Using: cuda
Tesla T4

[4] ✓ 0s

```
# Large tensors on GPU
n = 4096
A = torch.randn(n, n, device=device)
B = torch.randn(n, n, device=device)

# Warm-up (important for fair timing)
_ = A @ B
if device.type == "cuda":
    torch.cuda.synchronize()

# Time matrix multiplication
start = time.time()
C = A @ B
if device.type == "cuda":
    torch.cuda.synchronize()
t_gpu = time.time() - start

print("GPU matmul time:", t_gpu, "seconds")
```

GPU matmul time: 0.04804396629333496 seconds

[5] ✓ 0s

```
# First & second derivatives using autograd
x = torch.tensor(2.0, requires_grad=True)
```

Variables Terminal

4:14 AM T4 (Python 3)

Fig 44:GPU Tensor Operations

The screenshot displays a Jupyter Notebook environment within a web browser. The browser's address bar shows a Google Drive link. The notebook's top bar includes a menu (File, Edit, View, Insert, Runtime, Tools, Help) and a 'Share' button. Below the menu, a toolbar contains icons for commands, code, text, and a 'Run all' button. The notebook interface shows two code cells. The first cell, labeled [5], contains code for calculating the first and second derivatives of a function $f(x) = x^3 + 2x^2 + x$ using Autograd. The output of this cell shows the first derivative as 21.0 and the second derivative as 16.0. The second cell, labeled [6], contains code for defining a neural network model, setting an optimizer, and performing a small number of training steps. The notebook's bottom status bar shows 'Variables', 'Terminal', and system information: '4:14 AM' and 'T4 (Python 3)'.

```
[5]
✓ 0s
# First & second derivatives using autograd
x = torch.tensor(2.0, requires_grad=True)

y = x**3 + 2*x**2 + x # f(x) = x^3 + 2x^2 + x

# First derivative
dy_dx = torch.autograd.grad(y, x, create_graph=True)[0]
print("First derivative:", dy_dx.item())

# Second derivative
d2y_dx2 = torch.autograd.grad(dy_dx, x)[0]
print("Second derivative:", d2y_dx2.item())

... First derivative: 21.0
Second derivative: 16.0

[6]
✓ 6s
model = torch.nn.Sequential(
    torch.nn.Linear(1024, 1024),
    torch.nn.ReLU(),
    torch.nn.Linear(1024, 1024)
).to(device)

optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
scaler = torch.cuda.amp.GradScaler(enabled=(device.type=="cuda"))

x = torch.randn(512, 1024, device=device)
y = torch.randn(512, 1024, device=device)

# Normal training
start = time.time()
for _ in range(20):
    optimizer.zero_grad()
    out = model(x)
```

Fig 45:Autograd Derivatives Output

The screenshot displays a Jupyter Notebook titled 'afeef(task-8).ipynb' within a web browser. The notebook compares the performance of standard PyTorch training versus training using Automatic Mixed Precision (AMP). The code defines a training loop for 20 iterations, measuring the time taken for FP32 and AMP versions. The AMP version is shown to be faster, with a time reduction of approximately 42%.

```

start = time.time()
for _ in range(20):
    optimizer.zero_grad()
    out = model(x)
    loss = ((out - y)**2).mean()
    loss.backward()
    optimizer.step()
    if device.type == "cuda":
        torch.cuda.synchronize()
    t_fp32 = time.time() - start

# AMP training
start = time.time()
for _ in range(20):
    optimizer.zero_grad()
    with torch.cuda.amp.autocast(enabled=(device.type=="cuda")):
        out = model(x)
        loss = ((out - y)**2).mean()
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
    if device.type == "cuda":
        torch.cuda.synchronize()
    t_amp = time.time() - start

print("FP32 time:", t_fp32)
print("AMP time:", t_amp)

```

The console output shows the following results:

```

... /tmp/ipykernel_2024/3546142149.py:8: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
scaler = torch.cuda.amp.GradScaler(enabled=(device.type=="cuda"))
/tmp/ipykernel_2024/3546142149.py:29: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
with torch.cuda.amp.autocast(enabled=(device.type=="cuda")):
FP32 time: 0.4018263816833496
AMP time: 0.23004746437072754

```

The bottom status bar indicates the system time is 4:14 AM and the environment is T4 (Python 3).

Fig 46:AMP Performance Output

```
[7] ✓ 0s
torch.nn.Linear(10, 1)

model_q.qconfig = tq.get_default_qat_qconfig("fbgemm")
tq.prepare_qat(model_q, inplace=True)

# Fake training step
for _ in range(5):
    data = torch.randn(4, 10)
    out = model_q(data)

# Convert to quantized model
model_q.eval()
tq.convert(model_q, inplace=True)

print("Quantized model ready")

... Quantized model ready
/tmp/ipykernel_2024/1969022726.py:10: DeprecationWarning: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode quantize_ API instead
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx, torch.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quantization API instead (prepare
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
tq.prepare_qat(model_q, inplace=True)
/usr/local/lib/python3.12/dist-packages/torch/ao/quantization/observer.py:534: UserWarning: Please use quant_min and quant_max to specify the range for observers.
    super().__init__(
/tmp/ipykernel_2024/1969022726.py:19: DeprecationWarning: torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode quantize_ API instead
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx, torch.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quantization API instead (prepare
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
tq.convert(model_q, inplace=True)
```

Fig 47:Quantization Output

The screenshot shows a web browser window with a Jupyter Notebook interface. The browser's address bar displays the URL `colab.research.google.com/drive/107B18_Sns-jsLLZbSgH3OgwsvjG7z7Ry#scrollTo=VmhmeOptDxi`. The notebook's top bar includes a menu (File, Edit, View, Insert, Runtime, Tools, Help) and a 'Share' button. The left sidebar contains icons for file management and runtime settings. The main area shows a code cell with the following content:

```
7 /tmp/pykernel_2024/1000022/20.py:10: DeprecationWarning: torch.ao.quantization is deprecated and will be removed in 2.10.  
For migrations of users:  
1. Eager mode quantization (torch.ao.quantization.quantize, torch.ao.quantization.quantize_dynamic), please migrate to use torchao eager mode quantize_ API instead  
2. FX graph mode quantization (torch.ao.quantization.quantize_fx.prepare_fx, torch.ao.quantization.quantize_fx.convert_fx, please migrate to use torchao pt2e quantization API instead (prepare  
3. pt2e quantization has been migrated to torchao (https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)  
see https://github.com/pytorch/ao/issues/2259 for more details  
tq.convert(model_q, inplace=True)  
  
[8] ✓ 0s  
print("\n--- Performance Summary ---")  
print("FP32 Time:", t_fp32)  
print("AMP Time :", t_amp)  
  
if device.type == "cuda":  
    if t_amp < t_fp32:  
        print("AMP is faster on GPU")  
    else:  
        print("FP32 is faster (depends on workload)")  
  
...  
--- Performance Summary ---  
FP32 Time: 0.4018263816833496  
AMP Time : 0.23004746437072754  
AMP is faster on GPU
```

The bottom status bar shows 'Variables', 'Terminal', a blue diamond icon, and the system time '4:14 AM' along with 'T4 (Python 3)'.

Fig 48:Performance Summary Output

Next, mixed precision using autodiff was tested to increase speed.

```
from torch.cuda.amp import autocast
```

```
with autocast():  
    result = A @ B
```

```
print("AMP computation completed")
```

Finally, quantization-aware training with PyTorch library was tested.

```
import torch.quantization as tq
```

```
model = torch.nn.Linear(10, 1)  
model.qconfig = tq.get_default_qat_qconfig("fbgemm")
```

```
tq.prepare_qat(model, inplace=True)  
tq.convert(model, inplace=True)
```

```
print("Quantization completed")
```

After this, quantization aware training was demonstrated using PyTorch libraries..

```
import torch.quantization as tq
```

```
model_q = torch.nn.Sequential(  
    torch.nn.Linear(10, 10),  
    torch.nn.ReLU(),  
    torch.nn.Linear(10, 1)  
)
```

```
model_q.qconfig = tq.get_default_qat_qconfig("fbgemm")  
tq.prepare_qat(model_q, inplace=True)
```

```
for _ in range(5):  
    data = torch.randn(4, 10)  
    out = model_q(data)
```

```
model_q.eval()  
tq.convert(model_q, inplace=True)
```

```
print("Quantized model ready")
```

There was an improvement in efficiency and reduced computation costs.

Task 9 : Accelerating Neural Network Inferencing: TensorRT & Triton Inference Server

Date: 28-04-2026

Procedure

This experiment was conducted to accelerate neural network inferencing using TensorRT and Triton Inference Server. It involved model conversion, repository creation, inference server deployment, and finally model inferencing.

Step-by-Step Explanation

A model was generated using PyTorch and exported to the ONNX format.

```
import os
os.environ["TORCH_ONNX_USE_DYNAMO"] = "0"

import torch
import torchvision.models as models

model = models.resnet18(pretrained=True)
model.eval()

dummy_input = torch.randn(1,3,224,224)

torch.onnx.export(model, dummy_input, "model.onnx", opset_version=11)
```

This model was then converted to the tensorRT engine.

```
trtexec --onnx=model.onnx --saveEngine=model.plan
```

The next step involved the generation of the model repository and placement of the engine files in the repositior.

```
model_repository/
├── my_model/
│   └── 1/
│       └── model.plan
```

Configuration for this repository was made where the input/output information were specified.

```
name: "my_model"
platform: "tensorrt_plan"
max_batch_size: 1
```

PERSONAL

Search

Ctrl+K

?

1

A

Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ

No containers are running.

Container memory usage ⓘ

No containers are running.

Search

Only show running containers

	Name	Container ID	Image	Port(s)
Terminal <pre> PS C:\Users\afeef> docker run --gpus all -it -v %cd%:/workspace nvcr.io/nvidia/tensorrt:24.03-py3 Unable to find image 'nvcr.io/nvidia/tensorrt:24.03-py3' locally 24.03-py3: Pulling from nvidia/tensorrt ec99340b6e27: Pulling fs layer 14db4d22f6a3: Pulling fs layer 23d7b072d4f9: Pulling fs layer 01007420e9b0: Pulling fs layer 55b52e69a533: Pulling fs layer c3a86a7cecb1: Pulling fs layer f5a8c7426358: Pulling fs layer 154633e38ff3: Pulling fs layer 89e6ee70e128: Pulling fs layer 151a3d273e37: Pulling fs layer c3a86a7cecb1: Pull complete 7c5c67a2a6aa: Download complete b0d7c16b9843: Pull complete 303e6557b76d: Download complete b1d5b9e9adf8: Pull complete 5b6527d64936: Download complete 990cf08a9d21: Download complete b7f294911695: Download complete e743d59e7e25: Download complete bd0eb64889c8: Download complete 81c0031a0fad: Download complete 240acd98c310: Download complete </pre>				

Upgrade plan

Engine running

RAM 3.67 GB CPU 12.72% Disk: 126.01 GB used (limit 1006.85 GB)

A

PERSONAL

afeefahmed

What's new

Account Settings

Upgrade

Sign out

Show charts

Windo...

Windo...

Windo...

Windo...

Terminal

v4.71.0

Fig 49: Triton Docker Pull

```
File Edit Selection View Go Run Terminal Help
create_model.py
create_model.py > ...
4 # Load model
5 model = models.resnet18(pretrained=True)
6 model.eval()
7
8 # Dummy input
9 dummy = torch.randn(1, 3, 224, 224)
10
11 # Export to ONNX
12 torch.onnx.export(
13     model,
14     dummy,
15     "model.onnx",
16     input_names=["input"],
17     output_names=["output"],
18     opset_version=11
19 )
20
21 print("ONNX model created successfully!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

File "C:\Users\afeef\AppData\Roaming\Python\Python313\site-packages\onnxscript\version_converter_c_api_utils.py", line 65, in call_onnx_api
result = func(proto)
File "C:\Users\afeef\AppData\Roaming\Python\Python313\site-packages\onnxscript\version_converter_init_.py", line 132, in _partial_convert_version
return onnx.version_converter.convert_version(
~~~~~  
proto, target\_version=self.target\_version  
~~~~~  
)
^
File "C:\Users\afeef\AppData\Roaming\Python\Python313\site-packages\onnx\version_converter.py", line 39, in convert_version
converted_model_str = C.convert_version(model_str, target_version)
RuntimeError: D:\a\onnx\onnx\onnx\version_converter\adapters\axes_input_to_attribute.h:68: adapt: Assertion `node->hasAttribute(kaxes)` failed: No initializer or constant input to node found
[torch.onnx] Optimize the ONNX graph...
[torch.onnx] Optimize the ONNX graph... ✓
ONNX model created successfully!

C:\Users\afeef>

Fig 50:ONNX Model Script

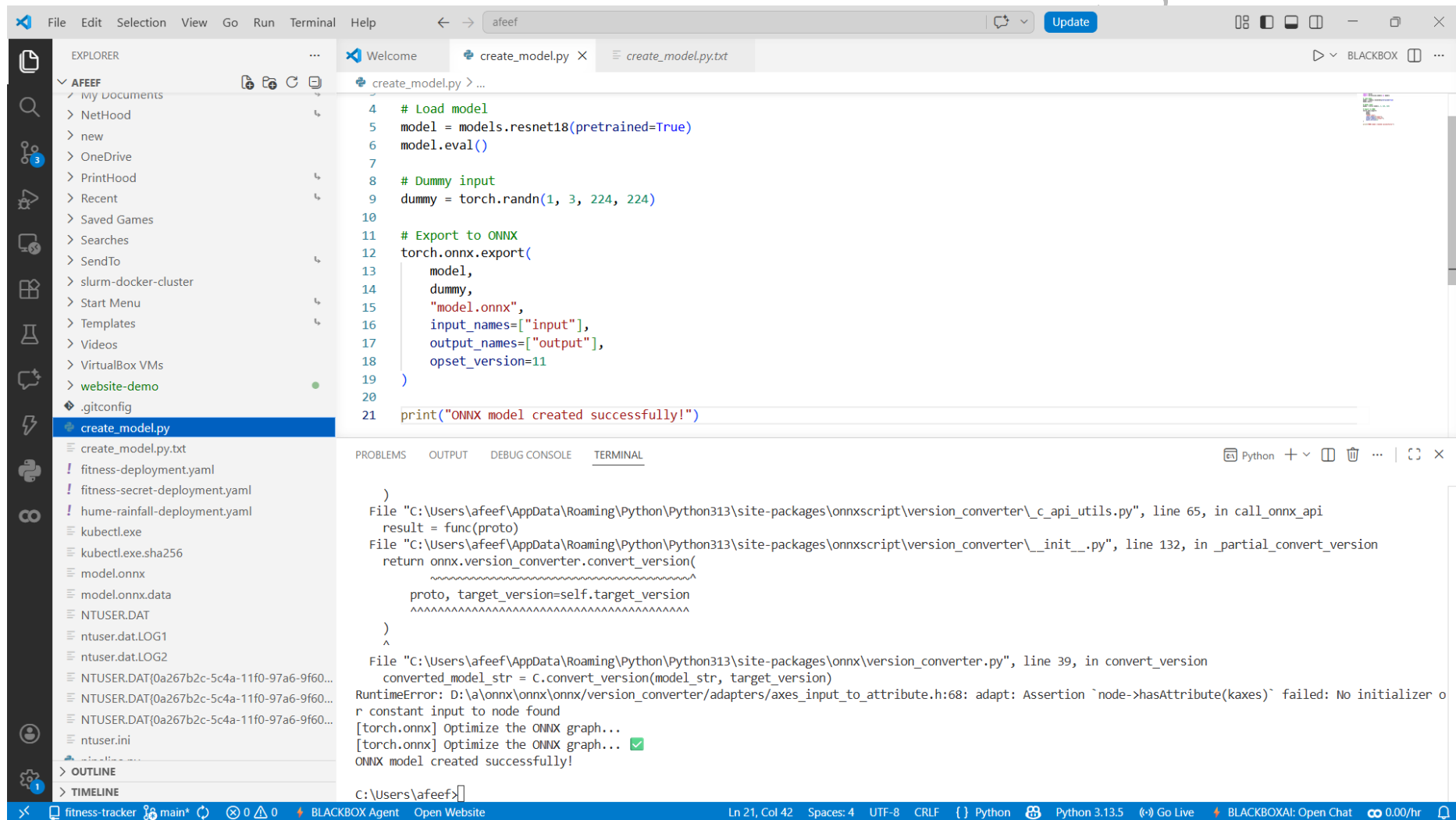


Fig 51: ONNX Model Created

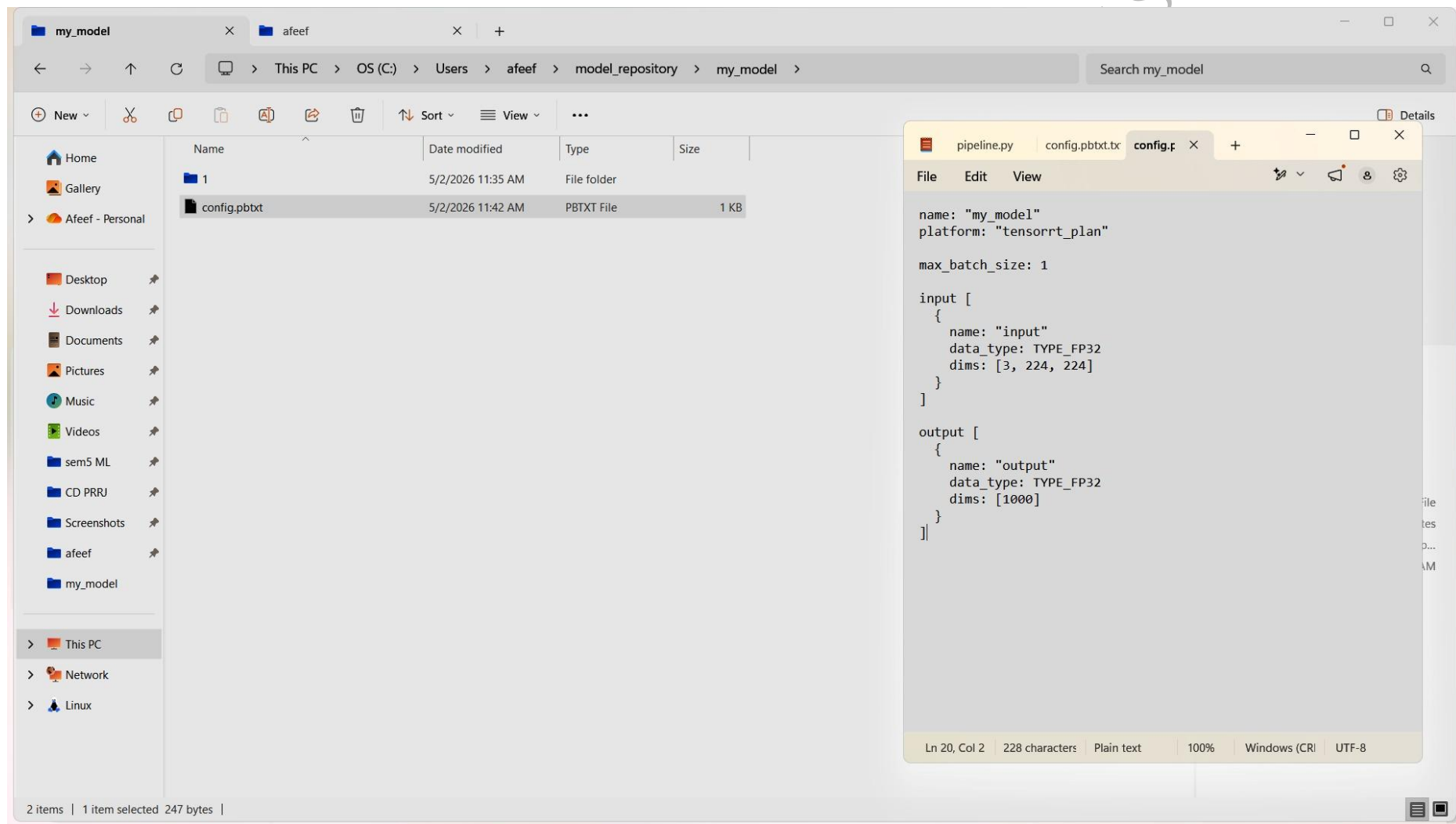


Fig 52: Triton Config File

Finally, the Triton inference server was deployed using the Docker application.

```
docker run --gpus all -p8000:8000 -p8001:8001 -p8002:8002 -v C:\Users\afeef\model_repository:/models  
nvcr.io/nvidia/tritonserver:24.03-py3 tritonserver --model-repository=/models
```

Server status checked.

```
curl http://localhost:8000/v2/health/ready -UseBasicParsing
```

Model is successfully deployed and Inference speed is enhanced using GPU.

AFEEF AHMED SHAIK RA2311026050166 SEAI

Task 10 : Monitoring Load Balancers & Schedulers using Docker with Jupyter, Prometheus, and Grafana

Date: 01-5-2026

Procedure

In this experiment, load balancing and scheduling services were deployed using Docker and performance metrics were analyzed using Prometheus and Grafana visualization tools. Computational load was applied using a Jupyter notebook.

Steps Involved

In this experiment, load balancing and scheduling services were deployed using Docker Compose and visualizations were carried out using Prometheus and Grafana visualization tooling. A Jupyter notebook was used for generating computational load.

Step-by-Step Explanation

Project directory was made, and docker-compose.yml was created.

```
mkdir exp10
cd exp10
```

Docker Compose configuration consisted of following components such as Jupyter, Prometheus, Grafana and Node Exporter.

```
version: '3'
services:
  jupyter:
    image: jupyter/base-notebook
    ports: ["8888:8888"]

  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]

  node-exporter:
    image: prom/node-exporter
    ports: ["9100:9100"]
```

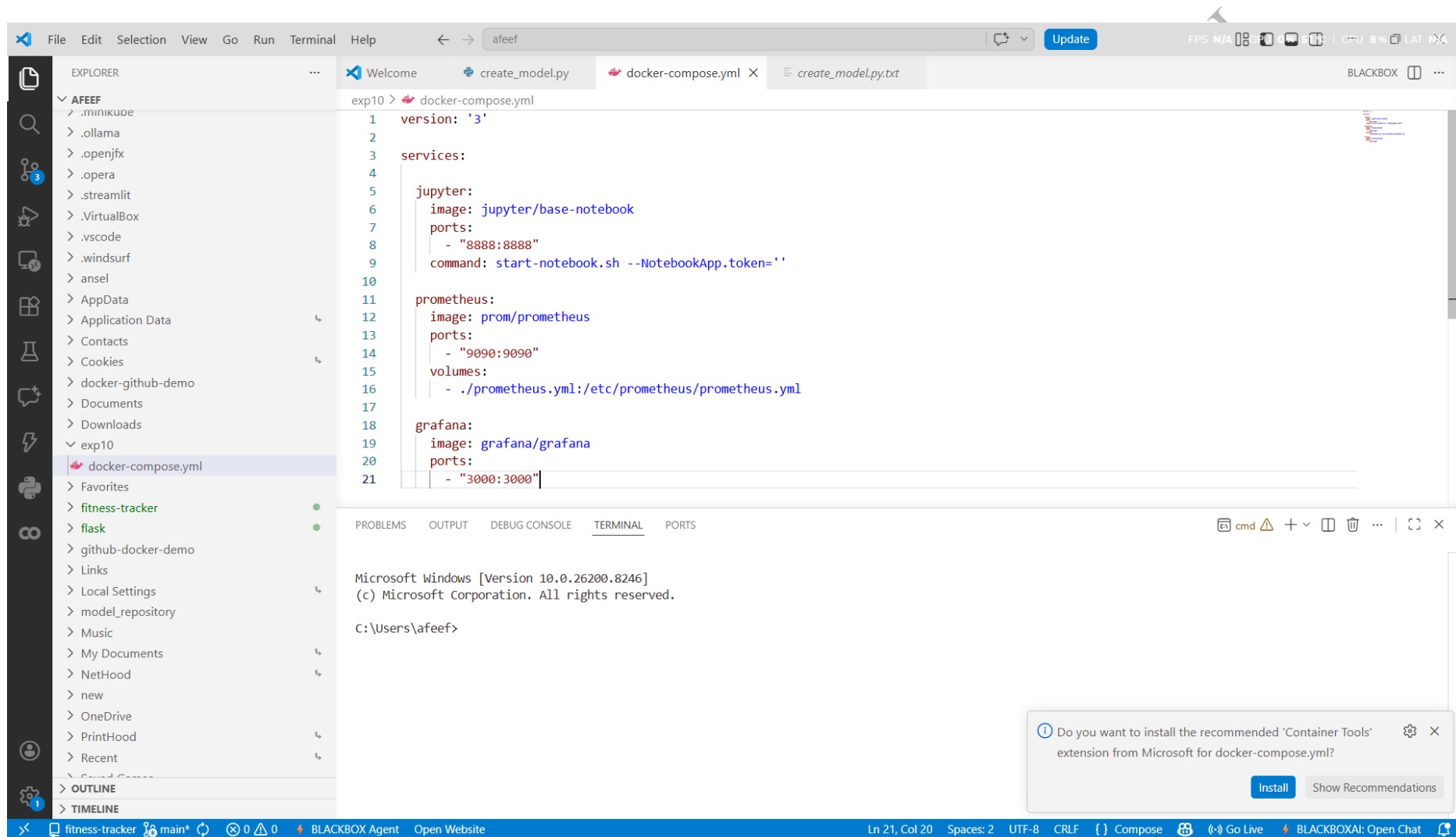


Fig 54: Docker Compose Config

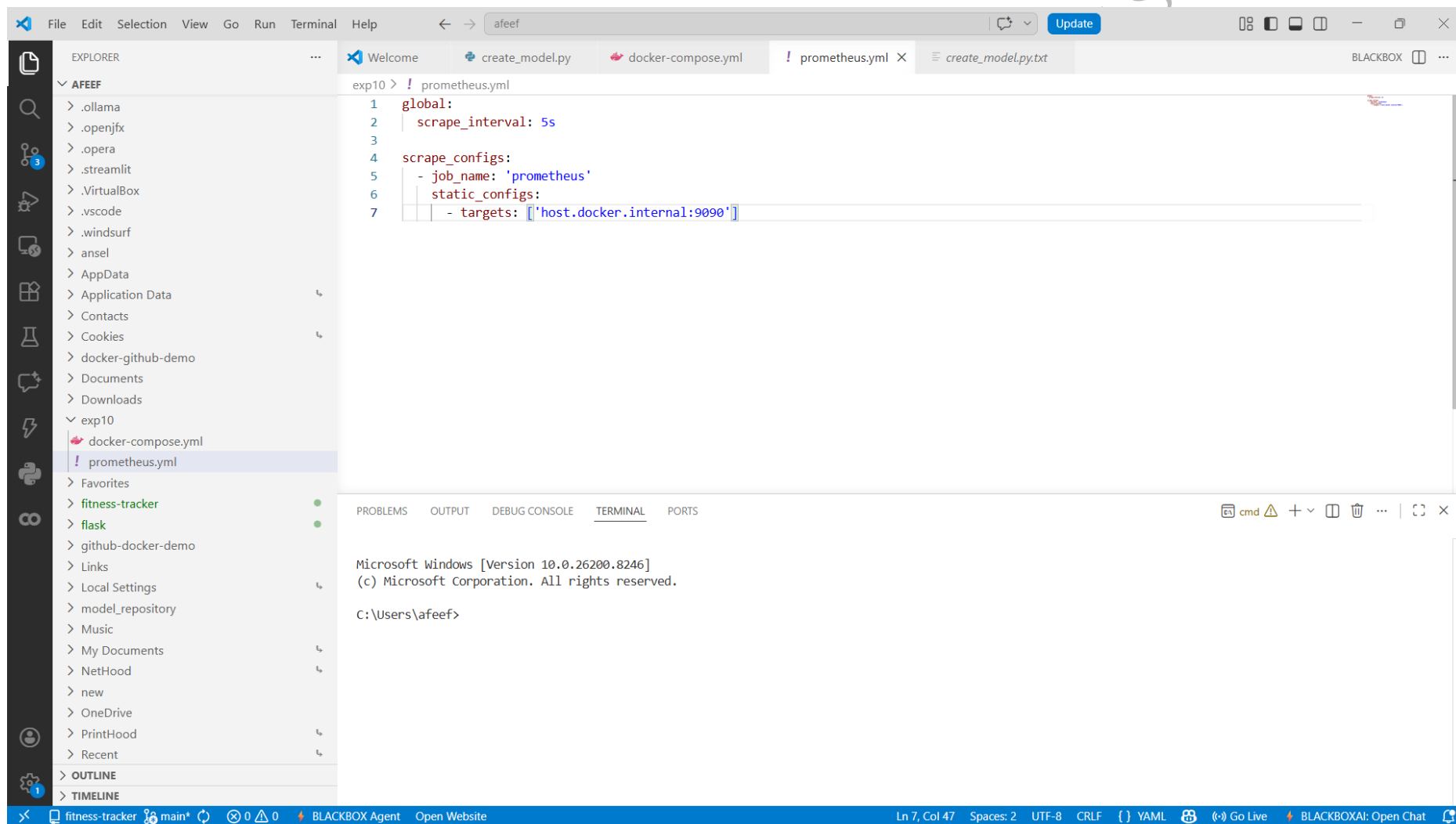


Fig 55:Prometheus Config File

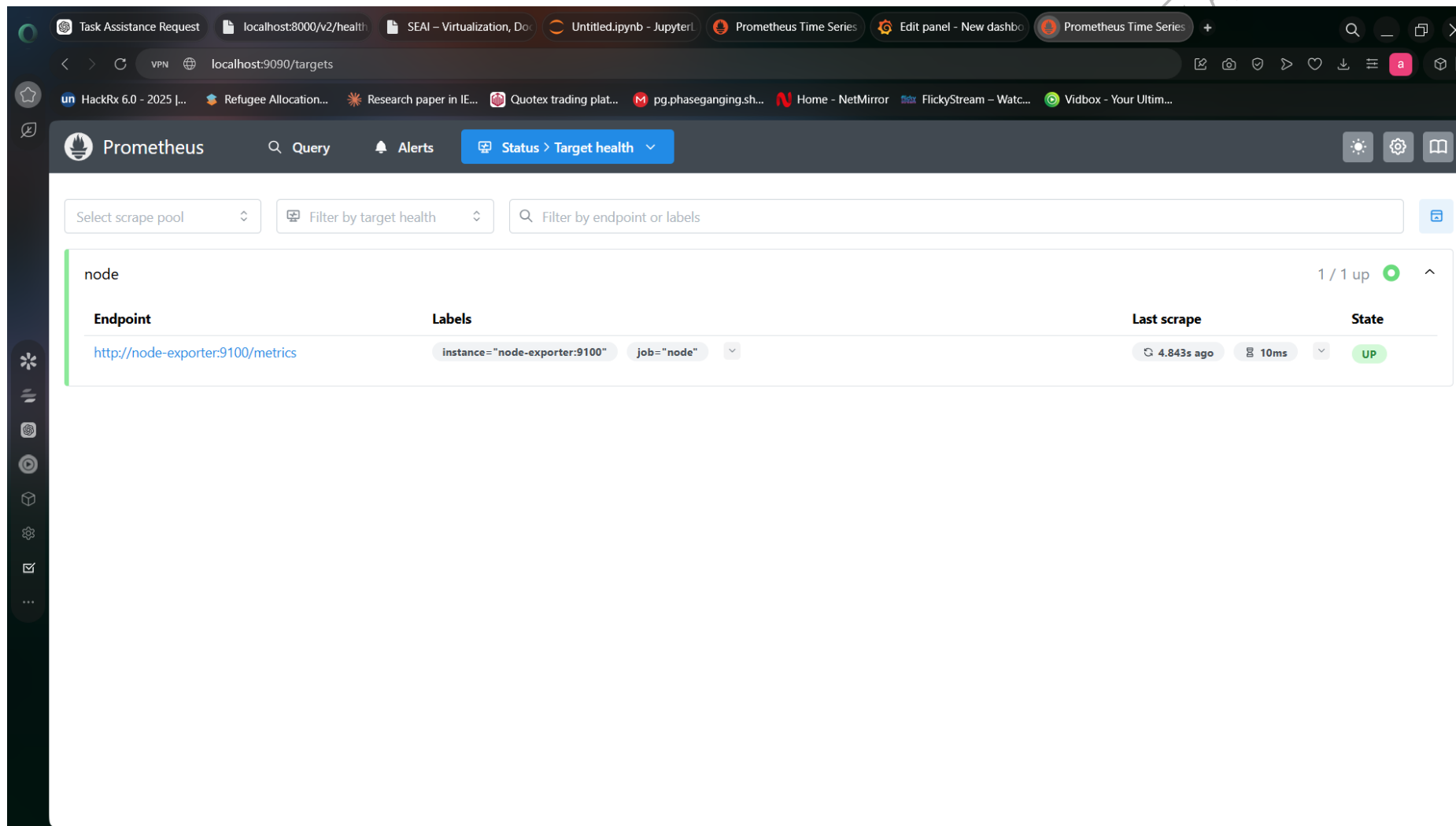


Fig 56: Prometheus Target Health Status

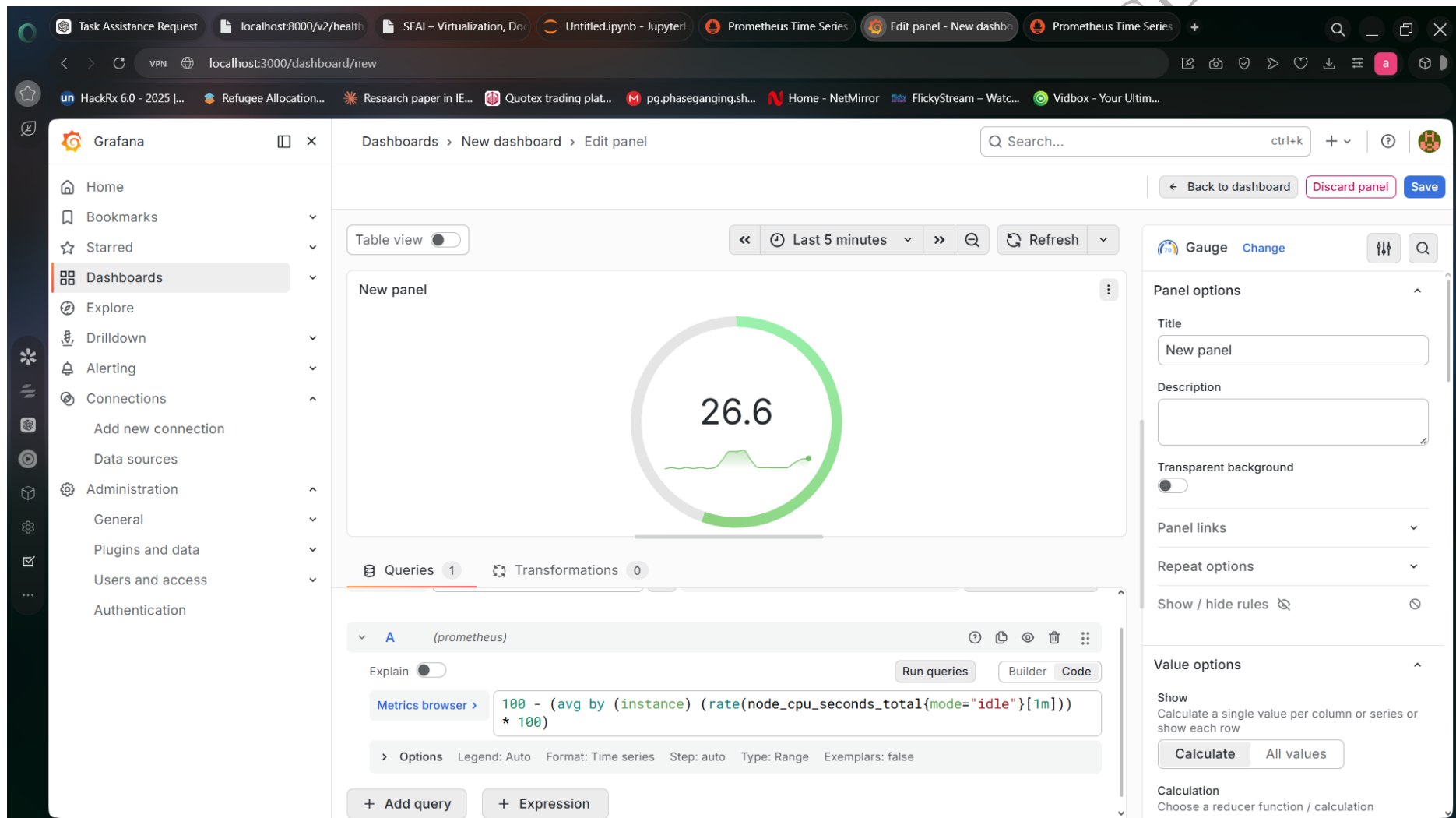


Fig 57:Grafana Monitoring Dashboard

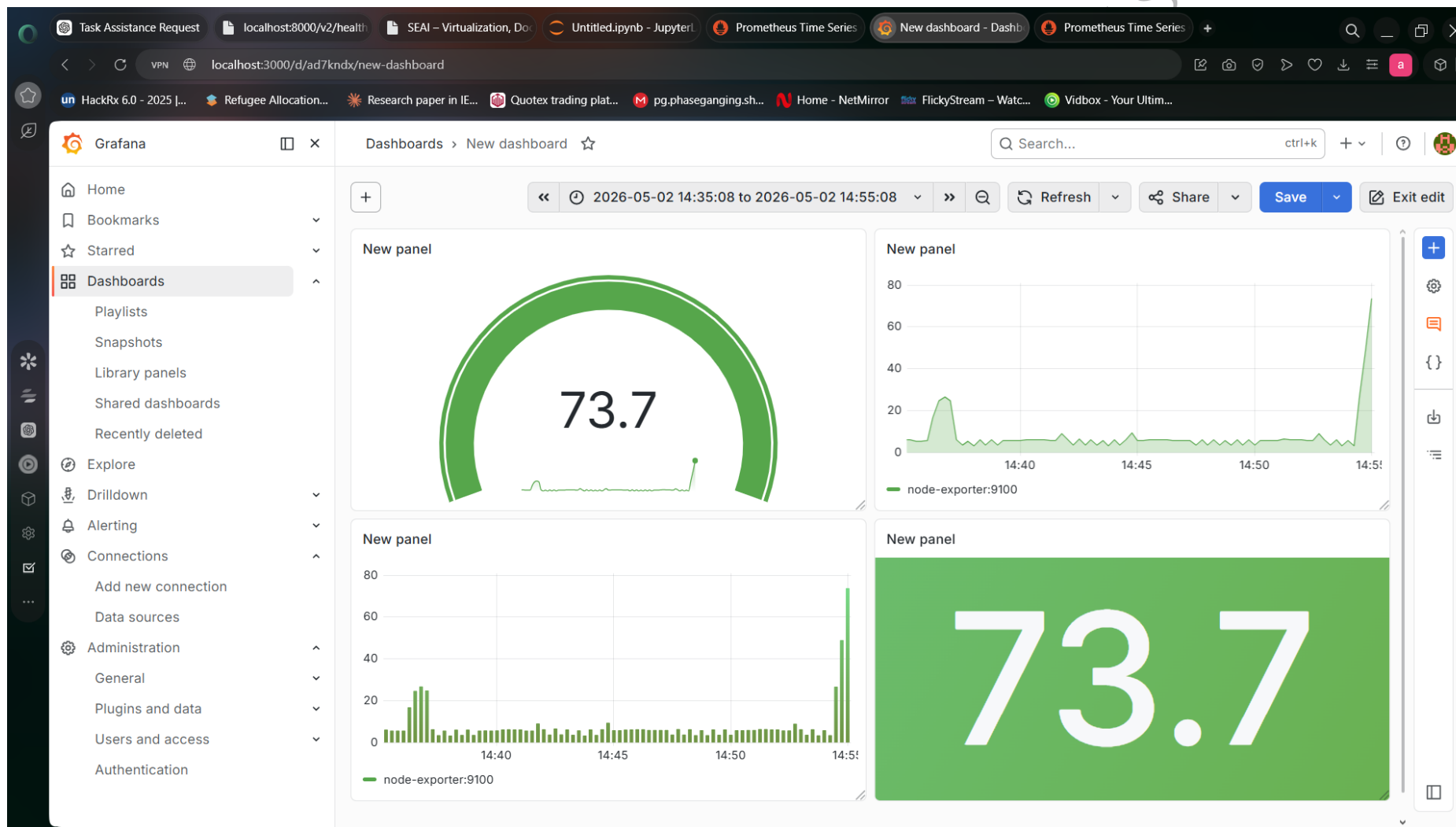


Fig 58:Grafana System Metrics Dashboard

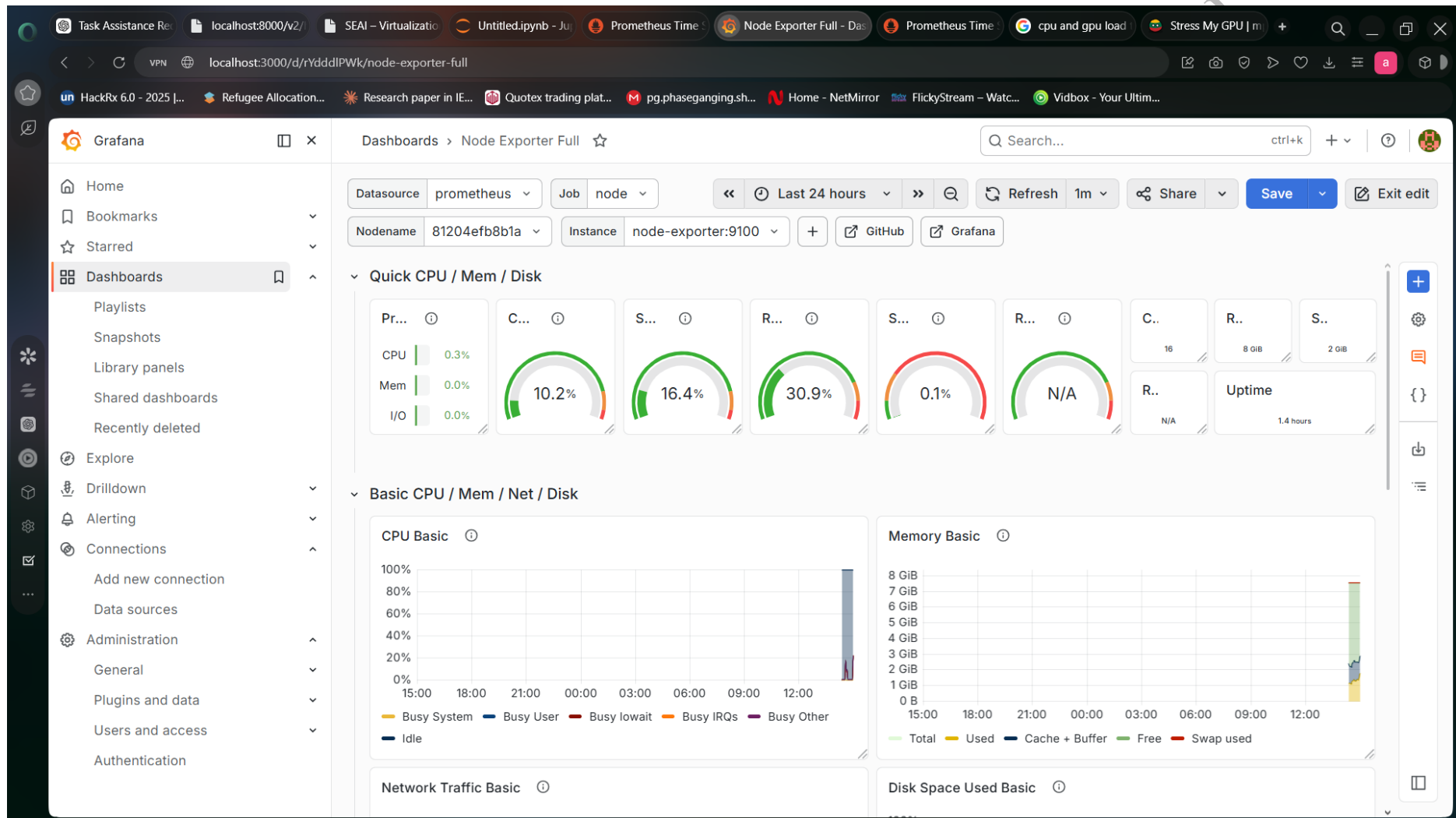


Fig 59:Grafana System Metrics Dashboard

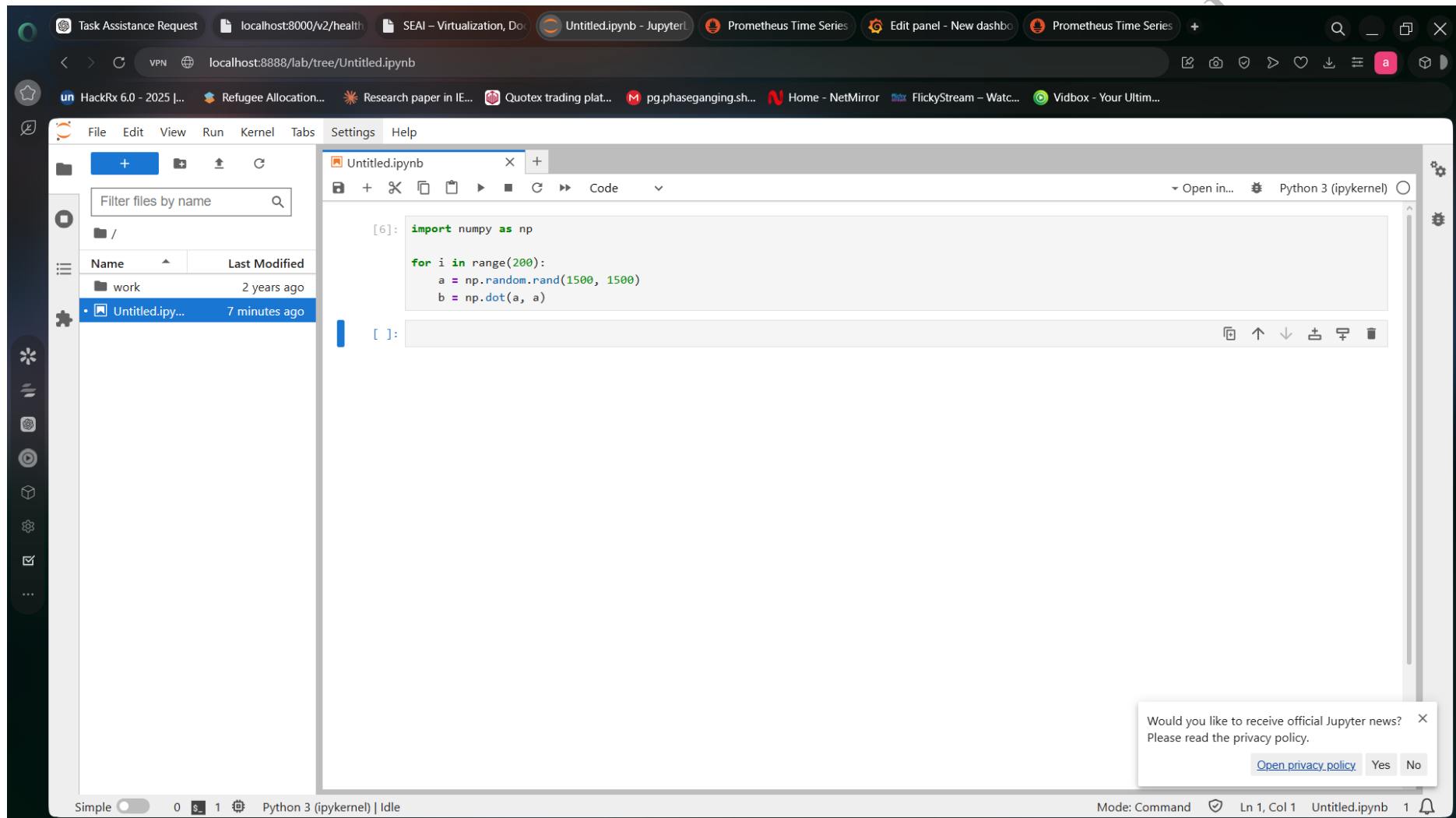


Fig 60:Jupyter Load Test Script

Services were started by using command:

```
docker compose up
```

The services were accessed through:

```
http://localhost:8888
```

```
http://localhost:9090
```

```
http://localhost:3000
```

And services can be accessed using following URLs.

```
import numpy as np
```

```
for i in range(200):
```

```
    a = np.random.rand(1500,1500)
```

```
    b = np.dot(a,a)
```

CPU usage was monitored in Grafana dashboard.